爬虫实战：从 0 到 1 做一个「采集 → 入库 → 分析 → 出报告」小程序

> 这是一份**能跑起来**的实战教程，不是概念罗列。
> 配套代码就在同一个文件夹里：爬虫实战_代码\（main.py + requirements.txt），**已经实测跑通。**

全局流程图（先看这张图）

【思维导图】以下为 Mermaid 源码，图形请在 Markdown 版查看

```
flowchart TD
 A["① 看懂网站<br/>F12 找数据位置"] --> B["② requests 发请求<br/>timeout 头部 状态码 编码"]
 B --> C["③ BeautifulSoup 解析<br/>CSS 选择器取字段"]
 C --> D["④ 翻页 while + next<br/>限速 sleep"]
 D --> E["⑤ 存 SQLite<br/>url 主键去重 UPSERT"]
 E --> F["⑥ pandas 分析<br/>mean groupby value_counts"]
 F --> G["⑦ matplotlib 出图<br/>中文字体 savefig"]
 G --> H["⑧ openpyxl 出 Excel<br/>python-docx 出 Word"]
 H --> I["⑨ 加 tenacity 重试<br/>loguru 日志 tqdm 进度"]
```

0. 你会做出什么

一条命令：

```
python main.py --pages 5
```

屏幕会显示：

```
抓取进度: 100%|██████████| 5/5 [00:10<00:00, 2.0s/页, 本页=20, 累计=100]
17:07:20 | SUCCESS | 抓取完成，共处理 100 条记录
17:07:20 | INFO | 共 100 本书，均价 34.96，最贵 57.25
17:07:21 | SUCCESS | Excel 已生成: out\books.xlsx
17:07:21 | SUCCESS | Word 已生成: out\report.docx

===== 汇总 =====
总数量 100
平均价格 34.96
最高价格 57.25
最低价格 12.84
平均星级 3.05
有货比例 100.0
```

文件夹里多出 4 个产物：

产物	内容
data/books.db	SQLite 数据库（原始数据，可反复查询）
out/price.png	价格分布 + 星级分布图（中文标题）
out/books.xlsx	Excel 报表（概览 / 全部图书 / 按星级统计 三个 sheet）
out/report.docx	Word 分析报告（含表格和插图）

技术栈：requests（抓）→ BeautifulSoup（解析）→ sqlite3（存）→ pandas（分析）→ matplotlib（图）→ openpyxl/python-docx（报告）→ tenacity/loguru/tqdm（工程化）

练习站点：books.toscrape.com ——
这是官方专门给爬虫练习用的站点（书是假的、价格是假的、没有反爬），可以放心练。

1. 三十秒环境准备

```
# ① 建虚拟环境（每个项目都该这么做）
python -m venv .venv
.venv\Scripts\activate # Windows
# source .venv/bin/activate # macOS / Linux

# ② 装依赖
pip install -r requirements.txt
# 国内加速：加 -i https://pypi.tuna.tsinghua.edu.cn/simple

# ③ 跑起来
python main.py --pages 2
```

项目结构（就两个文件，别怕）：

```
爬虫实战_代码/
├── main.py ← 全部逻辑都在这（约 300 行，分 9 个小节）
├── requirements.txt ← 依赖清单
├── data/ ← 自动创建：books.db + spider.log
└── out/ ← 自动创建：png / xlsx / docx
```

为什么用虚拟环境：爬虫的依赖更新很快，全局装容易和别的项目打架。见《常用第三方库速查》1.2 节。

2. 第 ① 步：先看懂目标网站（最容易被跳过，但最重要）

别急着写代码。 先花 5 分钟搞清楚"数据在哪、长什么样"。

1. 浏览器打开 https://books.toscrape.com/
2. 按 **F12** 打开开发者工具 → 切到 **Elements（元素）** 面板
3. 点左上角的"小箭头"图标，去点页面上的一本书的标题

你会看到这样的结构：

```
<article class="product_pod">
 <h3><a href="catalogue/a-light-in-the-attic_1000/index.html" title="A Light in the Attic">A Light in th
    e Attic</a></h3>
 <p class="star-rating Three"></p> <!-- 星级藏在 class 里！ -->
 <div class="product_price">
 <p class="price_color">£51.77</p> <!-- 价格带英镑符号 -->
 <p class="instock availability">In stock</p>
 </div>
</article>
```

这一步要产出三个结论（写在纸上或注释里）：

1. **一页有多少条** → 20 条，外层容器是 article.product_pod
2. **要抓哪些字段、分别在哪** → 标题在 h3 a、价格在 p.price_color、库存 p.instock、星级在 p.star-rating 的 **class** 里
3. **怎么翻页** → 页面底部有 <li class="next">next

> ※ **新手最大的误区**：不看页面结构就开始写 find，写十行删九行。
> 先看清结构，代码只有几行。

3. 第 ② 步：发请求（requests）

最小可用版本：

```
import requests

url = "https://books.toscrape.com/catalogue/page-1.html"
resp = requests.get(url, timeout=15)
print(resp.status_code) # 200
print(resp.text[:300]) # 看前 300 个字符
```

但能跑 ≠ 专业。 一个"不容易出事"的请求必须做这 4 件事：

```
HEADERS = {"User-Agent": "StudyBot/1.0 (learning crawler; for practice site only)"}

resp = requests.get(url, headers=HEADERS, timeout=15)

if resp.status_code == 200:
 resp.encoding = resp.apparent_encoding # ① 防中文乱码
 html = resp.text
```

必做	为什么
timeout=15	没超时，遇到卡住的连接**程序会永久挂住**
headers=...	默认 UA 是 python-requests/2.x，很多站点会直接拒绝；写明身份也更礼貌
检查 status_code	404/403 时 resp.text 是错误页，直接解析会得到空结果，很难排查
apparent_encoding	中文站常是 GBK，用默认编码会得到一堆乱码

常见状态码

- 200 正常
- 403 被拒绝（多半是 UA 或 IP 的问题）
- 404 页面不存在（检查 URL 拼写）
- 429 请求太频繁（**必须降速**，这是服务器在警告你）
- 5xx 服务器出错（可以稍后重试）

4. 第 ③ 步：解析 HTML（BeautifulSoup）

```
from bs4 import BeautifulSoup
from urllib.parse import urljoin

RATING_MAP = {"One": 1, "Two": 2, "Three": 3, "Four": 4, "Five": 5}

def parse_page(html, page_url):
 soup = BeautifulSoup(html, "lxml") # lxml 比 html.parser 快
 books = []

 for card in soup.select("article.product_pod"): # ① 遍历每一本书
 title_tag = card.select_one("h3 a")
 price_tag = card.select_one("p.price_color")
 stock_tag = card.select_one("p.instock.availability")
 rating_tag = card.select_one("p.star-rating")

 if not (title_tag and price_tag): # ② 结构不对就跳过
 continue

 raw_price = price_tag.get_text(strip=True) # "£51.77"
 price = float("".join(c for c in raw_price
 if c.isdigit() or c == ".")) # → 51.77

 stars = 0 # ③ class 里找星级
 if rating_tag:
 for cls in rating_tag.get("class", []):
 if cls in RATING_MAP:
 stars = RATING_MAP[cls]

 books.append({
 "title": title_tag.get("title") or title_tag.get_text(strip=True),
 "price": price,
 "rating": stars,
 "in_stock": 1 if stock_tag and "In stock" in stock_tag.get_text() else 0,
 "url": urljoin(page_url, title_tag["href"]), # ④ 相对链接转绝对
```

```
 })
 return books
```

逐段解释

① soup.select("article.product_pod")

CSS 选择器，article.product_pod 意思是"标签是 article 且 class 里有 product_pod"。返回**列表**，所以能 for 遍历。select_one 只取第一个。

② 为什么要 if not (...) 判断

网页随时可能改版、某条数据可能缺字段。**直接** title_tag.get_text() **会在数据缺失时抛 AttributeError 让整个程序崩掉**，一条脏数据毁掉一整轮抓取，很不划算。习惯是"缺了就跳过，并记日志"。

③ 星级为什么这么麻烦

<p class="star-rating Three"> —— 星级不是文字，而是写在 **class 名字**里（One~Five）。

所以取 tag["class"]（返回列表如 ["star-rating", "Three"]），逐个比对映射表。

这类"数据藏在属性里"的情况非常常见，比如 。

④ urljoin 一定要会

页面上写的是**相对路径** catalogue/xxx/index.html，直接存进数据库没法用。

urljoin("https://books.toscrape.com/catalogue/page-1.html", "catalogue/xxx/index.html") 会正确拼成完整 URL。

新手常犯的错：手动写 BASE + href，结果多一个斜杠、少一层目录，全 404。

CSS 选择器速查

写法	含义	例子
tag	标签名	select("a")
.cls	class	select(".price_color")
#id	id	select("#main")
a b	后代（任意层级）	select("div.product_price p")
a > b	直接子元素	select("ul > li")
[attr]	有某属性	select("a[href]")
[attr=值]	属性等于	select("a[title='X']")

取内容的三兄弟：tag.get_text(strip=True) 取纯文本、tag["href"] 取属性、tag.get("title") 取属性（没有返回 None 而不报错）。

5. 第 ④ 步：翻页（while + "下一页"按钮）

```
def next_page_url(html, page_url):
 soup = BeautifulSoup(html, "lxml")
 nxt = soup.select_one("li.next a")
 return urljoin(page_url, nxt["href"]) if nxt else None # 没有就返回 None

url, page = START_URL, 0
while url and page < max_pages: # ← 两个退出条件，缺一不可
 page += 1
 html = fetch(url)
 rows = parse_page(html, url)
 save_books(conn, rows)
 time.sleep(1.0) # 限速
 url = next_page_url(html, url) # 更新为下一页
```

三个坑

1. **最后一页没有"next"按钮** → select_one 返回 None → nxt["href"] 会 TypeError。

正确做法：用 if nxt else None 让 while 自然结束。

2. **死循环**：如果"下一页"的链接永远指向自己（有的站分页写错），while url 会转到天荒地老。

所以一定要加 page < max_pages 这个上限兜底。

3. **不给服务器喘气**：一秒钟发几十个请求 = 事实上的 DoS，会被封 IP，也很不道德。

本项目默认每页停 1 秒（time.sleep(1.0)）。

> **翻页的另外两种常见形态**
> - **URL 有规律**：?page=1,2,3... → 直接 for i in range(1, 51) 拼 URL，不需要解析"下一页"
> - **滚不到底**：瀑布流、无限滚动 → 用 playwright 滚动加载，或直接找它背后的 **JSON 接口**（DevTools 的 Network 面板筛 Fetch/XHR）

6. 第 ⑤ 步：存起来（SQLite）

为什么不直接存 CSV?

	CSV	SQLite
追加写入	要读出来再写回去，容易写乱	INSERT 一行即可

	CSV	SQLite
去重	自己想办法	主键 UNIQUE + ON CONFLICT 自动搞定
查询	全读进内存	SELECT ... WHERE 直接查
中途崩了	文件可能写坏	有事务保护
依赖	无	无（**标准库自带**）

```
import sqlite3

conn = sqlite3.connect("data/books.db")
conn.execute("""
 CREATE TABLE IF NOT EXISTS books(
 url TEXT PRIMARY KEY, -- 用网址做主键，天然去重
 title TEXT NOT NULL,
 price REAL,
 rating INTEGER,
 in_stock INTEGER,
 crawled_at TEXT
 )""")
conn.commit()

rows = [(r["url"], r["title"], r["price"], r["rating"], r["in_stock"], now) for r in data]
conn.executemany("""
 INSERT INTO books(url, title, price, rating, in_stock, crawled_at)
 VALUES (?, ?, ?, ?, ?, ?)
 ON CONFLICT(url) DO UPDATE SET
 title=excluded.title, price=excluded.price,
 rating=excluded.rating, in_stock=excluded.in_stock,
 crawled_at=excluded.crawled_at
""", rows)
conn.commit()
```

三个要点

1. **url 当主键**：同一个 URL 只会有一行。**这样重复跑脚本也不会产生重复数据**，这是爬虫最实用的技巧之一。
2. **INSERT ... ON CONFLICT DO UPDATE（UPSERT）**：已存在就更新。想"已存在就跳过"用 `INSERT OR IGNORE`。
3. **问号占位符**：`executemany` 的第二个参数。**永远不要用 f-string 拼 SQL**，那是 SQL 注入的入口（虽然这里数据是自己抓的，但习惯要从第一天养成）。

看看数据对不对：

```
for row in conn.execute("SELECT title, price, rating FROM books ORDER BY price DESC LIMIT 5"):
 print(row)
```

7. 第 ⑥ 步：分析（pandas）

爬虫只负责"拿到数据"，**价值在分析**。数据进库后就是最普通的表格分析了：

```
import pandas as pd, sqlite3

conn = sqlite3.connect("data/books.db")
df = pd.read_sql_query("SELECT * FROM books", conn) # 一行读出整张表
conn.close()

# 清洗
df = df.dropna(subset=["title", "price"])
df["price"] = df["price"].astype(float)

# 统计
print(len(df)) # 多少本
print(df["price"].mean()) # 均价
print(df["rating"].value_counts()) # 各星级数量
print(df.groupby("rating")["price"].mean()) # 每个星级的均价
print(df.sort_values("price", ascending=False).head(5)[["title", "price"]])
```

这四行你会在 90% 的数据分析里用到：len / mean / value_counts / groupby。

8. 第 ⑦ 步：出图（matplotlib）

```
import matplotlib
matplotlib.use("Agg") # 不弹窗，只存文件（脚本/服务器必备）
import matplotlib.pyplot as plt

plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"] # 中文标题别变方框
plt.rcParams["axes.unicode_minus"] = False

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
axes[0].hist(df["price"], bins=20, color="#4f81bd", edgecolor="white")
axes[0].set_title("价格分布"); axes[0].set_xlabel("价格 (£)"); axes[0].set_ylabel("数量")

counts = df["rating"].value_counts().sort_index()
axes[1].bar(counts.index.astype(str), counts.values, color="#c0504d")
axes[1].set_title("星级分布")

plt.tight_layout()
plt.savefig("out/price.png", dpi=150, bbox_inches="tight") # 在 show 之前保存
plt.close(fig)
```

三个必踩的坑（我都踩过）

- **中文变方框** → 必须设 `font.sans-serif`（Windows 用 Microsoft YaHei，macOS 用 Arial Unicode MS）
- **保存的图是空的** → `plt.savefig` 必须写在 `plt.show()` **之前**（show 之后画布就清了）
- **Agg 是干嘛的** → 没装图形界面的机器（服务器）上默认后端会报错，Agg 表示"只画到文件"

9. 第 ⑧ 步：出报告（openpyxl + python-docx）

数据抓回来、分析完，最后要能"交出去"。

9.1 Excel：多个 sheet + 排版

```
import pandas as pd

with pd.ExcelWriter("out/books.xlsx", engine="openpyxl") as writer:
 pd.DataFrame(list(stats.items()), columns=["指标", "数值"]).to_excel(
 writer, sheet_name="概览", index=False) # sheet 1
 df.sort_values("price", ascending=False).to_excel(
 writer, sheet_name="全部图书", index=False) # sheet 2
 df.groupby("rating")["price"].agg(["count", "mean"]).round(2).to_excel(
 writer, sheet_name="按星级统计") # sheet 3
```

index=False 非常关键 —— 不然 Excel 里会多出一列没用的行号。

9.2 Word：标题 + 表格 + 插图

```
from docx import Document
from datetime import datetime

doc = Document()
doc.add_heading("图书数据采集分析报告", level=1)
doc.add_paragraph("生成时间：" + datetime.now().strftime("%Y-%m-%d %H:%M:%S"))

# 表格
table = doc.add_table(rows=1, cols=2)
table.style = "Table Grid"
table.rows[0].cells[0].text = "指标"
table.rows[0].cells[1].text = "数值"
for k, v in stats.items():
 cells = table.add_row().cells
 cells[0].text, cells[1].text = str(k), str(v)

# 项目符号列表
for _, row in top5.iterrows():
 doc.add_paragraph("%s —— £%.2f" % (row["title"], row["price"]), style="List Bullet")

# 插图
doc.add_picture("out/price.png")

doc.save("out/report.docx")
```

常见 style 值："List Bullet"（圆点）、"List Number"（编号）、"Intense Quote"（强调引用）。

10. 第 ⑨ 步：串起来

```
def main():
 ap = argparse.ArgumentParser(description="图书数据采集与分析")
 ap.add_argument("--pages", type=int, default=5, help="抓取页数（每页 20 本）")
 ap.add_argument("--skip-crawl", action="store_true", help="跳过抓取，只用已有数据出报告")
 args = ap.parse_args()

 if not args.skip_crawl:
 crawl(args.pages) # ① 抓

 df = load_data() # ② 读
 stats = summarize(df) # ③ 分析
 chart = make_chart(df) # ④ 画图
 make_excel(df, stats, ...) # ⑤ 出 Excel
 make_word(stats, ..., chart) # ⑥ 出 Word

if __name__ == "__main__":
 main()
```

--skip-crawl **是很有用的设计**：调整报告样式时不用重新爬一遍，几秒就能看效果。

完整代码在 爬虫实战_代码\main.py（约 300 行，分成 9 个小节，每节都有中文注释）。

11. 第 ⑩ 步：让它更健壮（这才是"能用的爬虫"）

11.1 失败自动重试（tenacity）

网络抖动、超时、偶发 5xx 都太常见了，**不要一失败就退出**：

```
from tenacity import retry, stop_after_attempt, wait_fixed, retry_if_exception_type

class RetryableError(Exception):
 """网络类错误，交给 tenacity 重试"""

@retry(stop=stop_after_attempt(3), wait=wait_fixed(2),
 retry=retry_if_exception_type(RetryableError), reraise=True)
def fetch(url):
 try:
 resp = requests.get(url, headers=HEADERS, timeout=15)
 except requests.RequestException as exc:
 raise RetryableError(str(exc)) from exc # 可重试的错误
 if resp.status_code >= 500:
 raise RetryableError("HTTP %d" % resp.status_code)
 if resp.status_code != 200:
 raise RuntimeError("HTTP %d" % resp.status_code) # 4xx 不重试
 return resp.text
```

关键设计：区分"**值得重试**"（超时、5xx）和"**重试也没用**"（404、403、解析失败）。全都重试只会浪费时间和被封。

11.2 日志（loguru）

别再用 print 调试了，日志能落盘、能分级、能带时间：

```
from loguru import logger
logger.add("data/spider.log", rotation="5 MB", encoding="utf-8")

logger.info("抓取第 {} 页: {}", page, url)
logger.success("抓取完成，共 {} 条", total)
try:
 ...
except Exception:
 logger.exception("这一页出问题了") # 自动附带完整堆栈
```

`rotation="5 MB"` 表示日志超过 5MB 自动换新文件，不会把磁盘写爆。

11.3 进度条（tqdm）

```
from tqdm import tqdm
bar = tqdm(total=max_pages, desc="抓取进度", unit="页")
while ...:
 bar.update(1)
 bar.set_postfix(本页=len(rows), 累计=total)
bar.close()
```

11.4 断点续爬

因为 `url` 是主键 + UPSERT，**直接重新跑就会自动"续爬"**（已抓的更新，没抓的补上），不需要额外的断点文件。

想更省流量，可以只抓"还没抓过的 URL"：

```
done = {row[0] for row in conn.execute("SELECT url FROM books")}
if book["url"] in done:
 continue # 跳过已抓过的
```

11.5 数据落盘策略

方式	适合
SQLite	结构化、要反复查、要增量更新 ← **本项目**
CSV / Excel	一次性导出给别人看
JSON / JSONL	字段不固定、要给别的程序读
MongoDB	文档结构差异大、数据量很大

12. 报错排查表（遇到问题先查这里）

报错 / 现象	原因	怎么办
`ModuleNotFoundError: No module named 'bs4'`	没装库 / 装到了别的环境	确认虚拟环境已激活，`pip install beautifulsoup4`
`requests.exceptions.ConnectTimeout`	网络慢或被墙	加大 timeout、加重试、检查代理
`SSLError`	证书问题	别用 `verify=False` 糊过去；升级 `certifi`
状态码 403	被拒绝（UA / 频率）	补 `User-Agent`、降速、检查是否违反条款
状态码 429	请求太频繁	**立刻降速**，加 `time.sleep`
`AttributeError: 'NoneType' object has no attribute ...`	`select_one` 没找到	打印 HTML 确认选择器；加 `if tag:` 判断
解析结果是空的	页面是 **JS 渲染**的	换 `playwright/selenium`，或找背后的 JSON 接口
中文乱码	编码不对	`resp.encoding = resp.apparent_encoding`
数据只有第一页	翻页逻辑没走通	打印 `next_page_url` 的返回值看看
图里中文是方框	没设字体	`plt.rcParams["font.sans-serif"] = ["Microsoft YaHei"]`
保存的图是空白	savefig 在 show 之后	把 savefig 挪到 show 之前
Word 里图片太大/太小	没设宽高	`doc.add_picture(path, width=Inches(6))`
`sqlite3.OperationalError: database is locked`	多个进程同时写	写的时候别开多个程序；用完 `conn.close()`

通用的排查三步：① 看报错的**最后一行**（错误类型）和**倒数第二行**（行号）；

② 在出错前 `print` 出关键变量（尤其是选择器的返回值和类型）；③ 用浏览器 DevTools 重新确认页面结构。

13. [!] 合规红线（必读，比技术更重要）

爬虫在中国是有明确法律边界的，**这几条必须记住**：

[√] 可以做

- 抓取**公开**的、不违反站点条款的数据

- 遵守 robots.txt
- 控制频率（本教程默认每页 1 秒）
- 用站点的**官方 API**（有 API 就别爬）
- 在专门的教学站点上练习（books.toscrape.com、quotes.toscrape.com、httpbin.org）

[×] 绝对不能做

- **绕过登录 / 付费墙 / 验证码** 去拿非公开数据
- **抓取个人信息**（手机号、身份证、住址、聊天记录）—— 可能触犯《个人信息保护法》《刑法》253 条之一
- **高频请求打挂对方服务** —— 可能构成《刑法》285/286 条的"破坏计算机信息系统"
- 抓来的数据**转卖、公开传播**
- 用爬虫做**刷单、抢票、水军**等

一句话判断法：

> 如果你要爬的东西，**"用浏览器正常点开、不登录、不付费就能看到"**，并且**你只少量、低频、不传播地用于学习**——那基本安全。
> 其他情况，**先查法律、先问授权。**

14. 进阶方向（学完这份，下一步往哪走）

方向	学什么	用什么
处理 JS 页面	等待、点击、滚动、截屏	playwright
工程化爬虫	队列、去重、并发、中间件	scrapy
大规模并发	async 并发请求	httpx + asyncio
反爬对抗（合规范围内）	代理池、请求头、频率控制	httpx、fake-useragent
数据存储升级	数据库、增量、分表	pymysql / pymongo
数据展示	交互式图表、看板	plotly / streamlit
定时运行	每天自动抓一次	schedule / 系统计划任务
自动化操作	抢不到的按钮就模拟点击	playwright / pyautogui

15. 自测题（做完就算掌握了）

1. 为什么 requests.get 一定要加 timeout？
2. select("h3 a") 和 select_one("h3 a") 有什么区别？
3. urljoin 解决的是什么问题？手动拼字符串会出什么错？
4. 为什么要用 url 当数据库主键？
5. INSERT OR IGNORE 和 INSERT ... ON CONFLICT DO UPDATE 有什么区别？各适合什么场景？
6. 为什么 plt.savefig 必须写在 plt.show() 之前？
7. 429 状态码在表达什么？你应该怎么做？
8. 页面用 JS 渲染时，requests 为什么抓不到数据？有哪两种解决办法？
9. 什么是"合理的请求频率"？怎么实现？
10. 举出三种**绝对不能爬**的数据类型。

> 答案都能在上面的正文里找到。做不出来就回去翻对应小节。

（本教程由 AI 助手整理，配套代码已实测运行通过；请仅用于合法合规的学习场景。）